Performance optimization in Go compiler & applications on s390x

Srinivas Pokala
Kiran M Vijay

Accelerating Goroutine ID access during database connectivity on s390x

Motivation:

- **What is a Goroutine?**
  - Lightweight thread's managed by Go runtime
  - Can run concurrently with thousands of other goroutines
  - Communicate via channels or shared memory
  - Scheduled by Go's **G-P-M runtime**, not tied to OS threads
- **Why Goroutine ID Matters?**
  - Tracing/Observability: Track execution paths and request flow
  - Debugging: Identify which goroutine caused a bug
  - Deadlock detection: Know which goroutine holds which lock
  - Profiling: Measure CPU/memory per goroutine

Problem – Delayed Goroutine Id Extraction Mechanism on s390x

- runtime.Stack()–based extraction is expensive.
  - Captures the entire goroutine stack → **produces large text output**
  - Requires allocating and parsing large stack-trace text
  - Allocates memory, causing GC pressure
  - Parses text to locate the "goroutine <id>" header
  - ~ 5577 ns per call to fetch Goroutine Id
- Latency in accessing databases (for eg: Cockroach DB) by Go applications
- Not designed for frequent or real-time use
  - Typically used only during panics or explicit diagnostics
- Unreliable under concurrency
  - Multiple goroutines panicking simultaneously can **interleave stack output**, making GID extraction incorrect or incomplete

Solution – Enable Fast Path Mechanism for Goroutine Id Extraction on s390x

Key Idea:

- On **s390x**, the Go runtime keeps the current goroutine's '**g' pointer** in a **dedicated register(R13)**
  - No need for Go runtime.Stack call's**.**
- **Approach:**
  - Use a tiny **assembly stub** to read the 'g' pointer from the dedicated register (R13).
  - Extract the **goid** field from the 'g' struct
  - No parsing, string manipulation, or runtime function calls needed.
  - No stack trace, no allocations.
  - Always ensures constant-time lookup.
  - Fallback mechanism for older go versions, ensuring portability.
  - **~1.87 ns** per call to fetch Goroutine Id

Benchmarking & Results

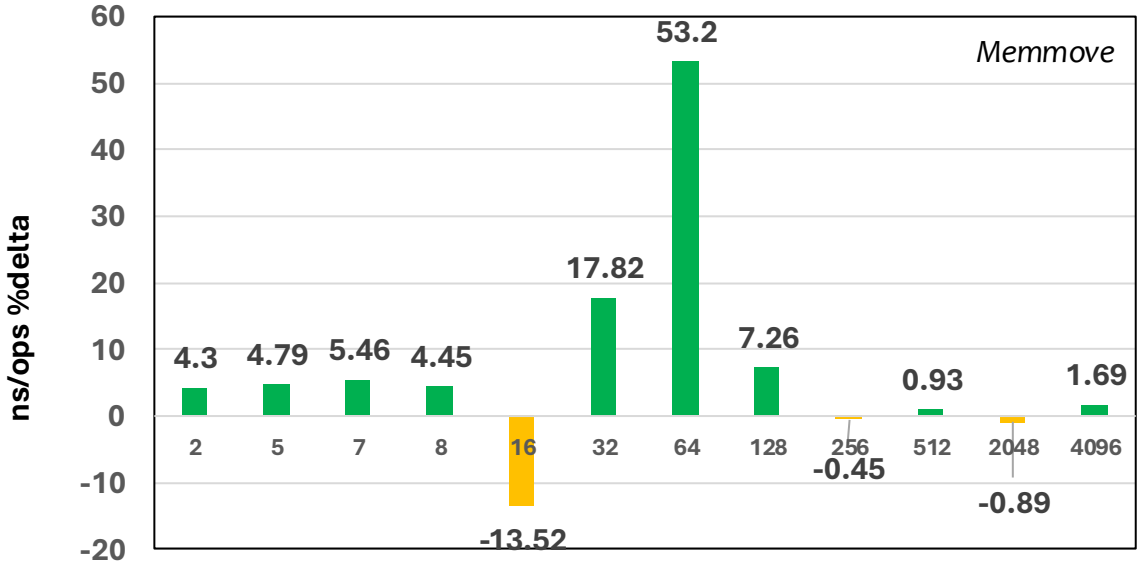
- Compared new fast-path against slow path goroutine ID extraction.
- Measured allocations, latency, CPU usage
- Achieved ~2900x performance improvement
- Benefitted with Highly concurrent and distributed database access such as CockroachDB, TiDb etc.
- Also, benefits deadlock-detection tools which can leverage goroutine-ID fetching mechanisms for its improved efficiency.

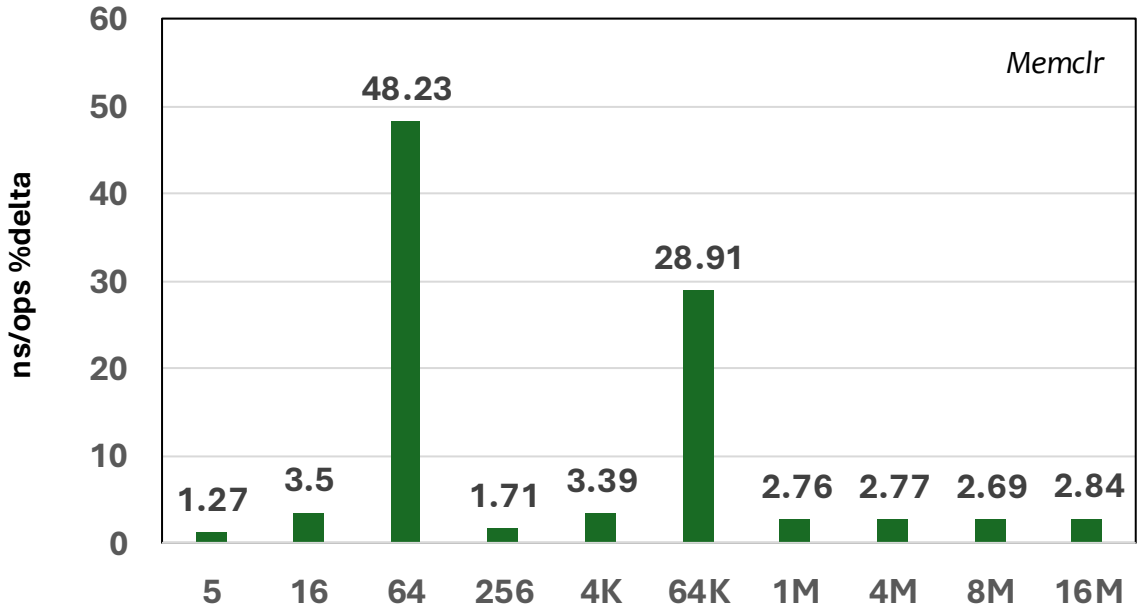
~2900x performance improvement

	s390x		x86-64
Benchmark Details	**Without patch (Slow-path)**	**With patch (Fast-path)**	
BenchmarkGet-128	5616 ns/op	1.921 ns/op	1.791 ns/op
BenchmarkGet-128	5642 ns/op	1.942 ns/op	1.663 ns/op
BenchmarkGet-128	5593 ns/op	1.807 ns/op	1.644 ns/op
BenchmarkGet-128	5520 ns/op	1.819 ns/op	1.637 ns/op
BenchmarkGet-128	5625 ns/op	1.948 ns/op	1.823 ns/op
BenchmarkGet-128	5541 ns/op	1.951 ns/op	1.826 ns/op
BenchmarkGet-128	5561 ns/op	1.787 ns/op	1.747 ns/op
BenchmarkGet-128	5557 ns/op	1.790 ns/op	1.840 ns/op
BenchmarkGet-128	5595 ns/op	1.944 ns/op	1.926 ns/op
BenchmarkGet-128	5520 ns/op	1.794 ns/op	1.723 ns/op

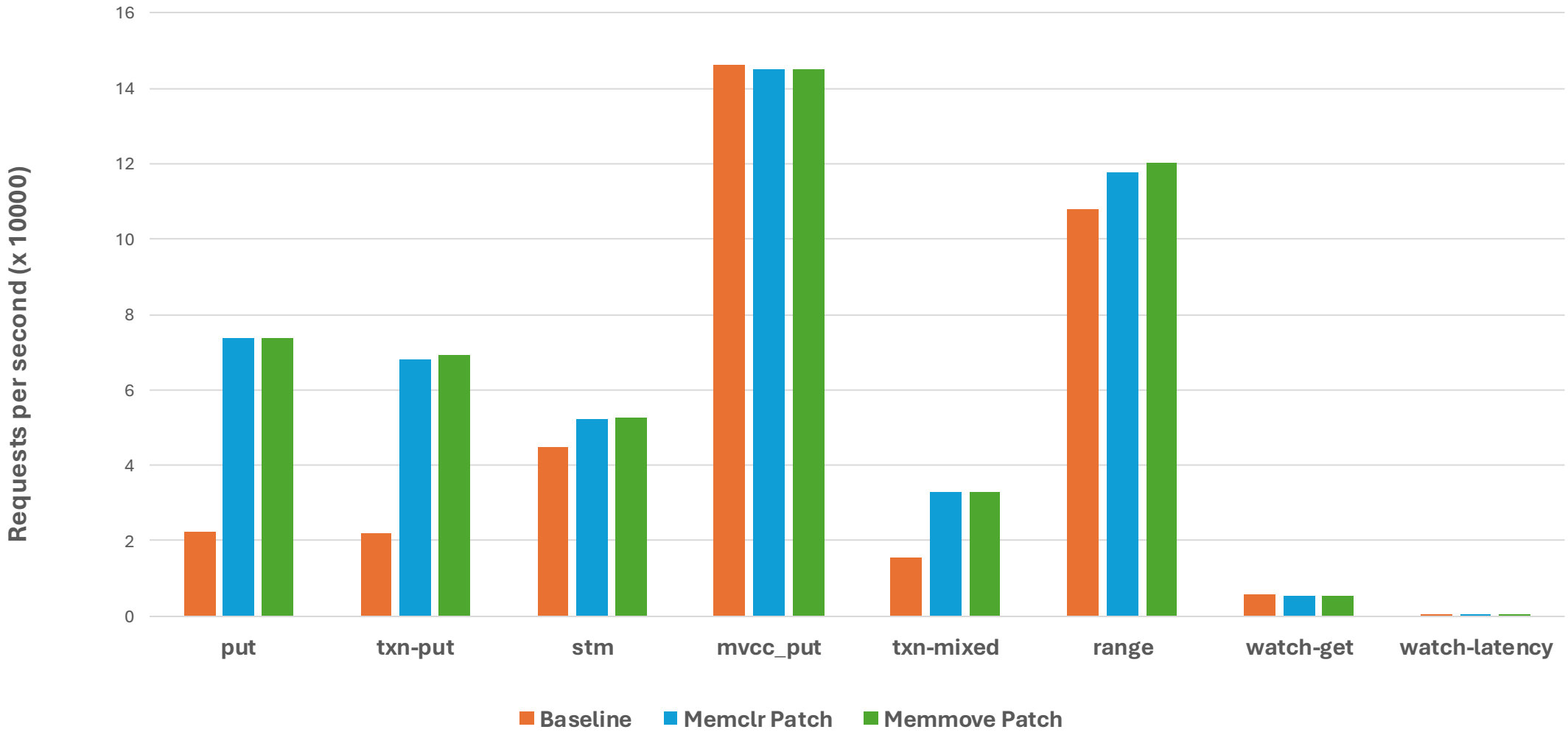
Optimized Memory Move & Clear Operations

- Use cases:
  - Object creation
  - Slice manipulation
  - Garbage collection, etc...
- Issue:
  - Database applications reported bottleneck in memclr and memmove on s390x.
- Optimizations:
  - Scalar to Vector instructions
  - Assembly level optimization such as Loop unrolling, Better branching & Instruction pipelining etc.
- Tools used:
  - Linux Perf
  - GDB & objdump
  - Go Test Suite Micro-benchmarking
- Result: Summary of performance improvement (Go micro-benchmarks)
  - Upto 50% for smaller data size (<= 256B)
  - Upto 30% for medium data size (<= 64KB)
  - Upto 4% for larger data size (> 64KB)



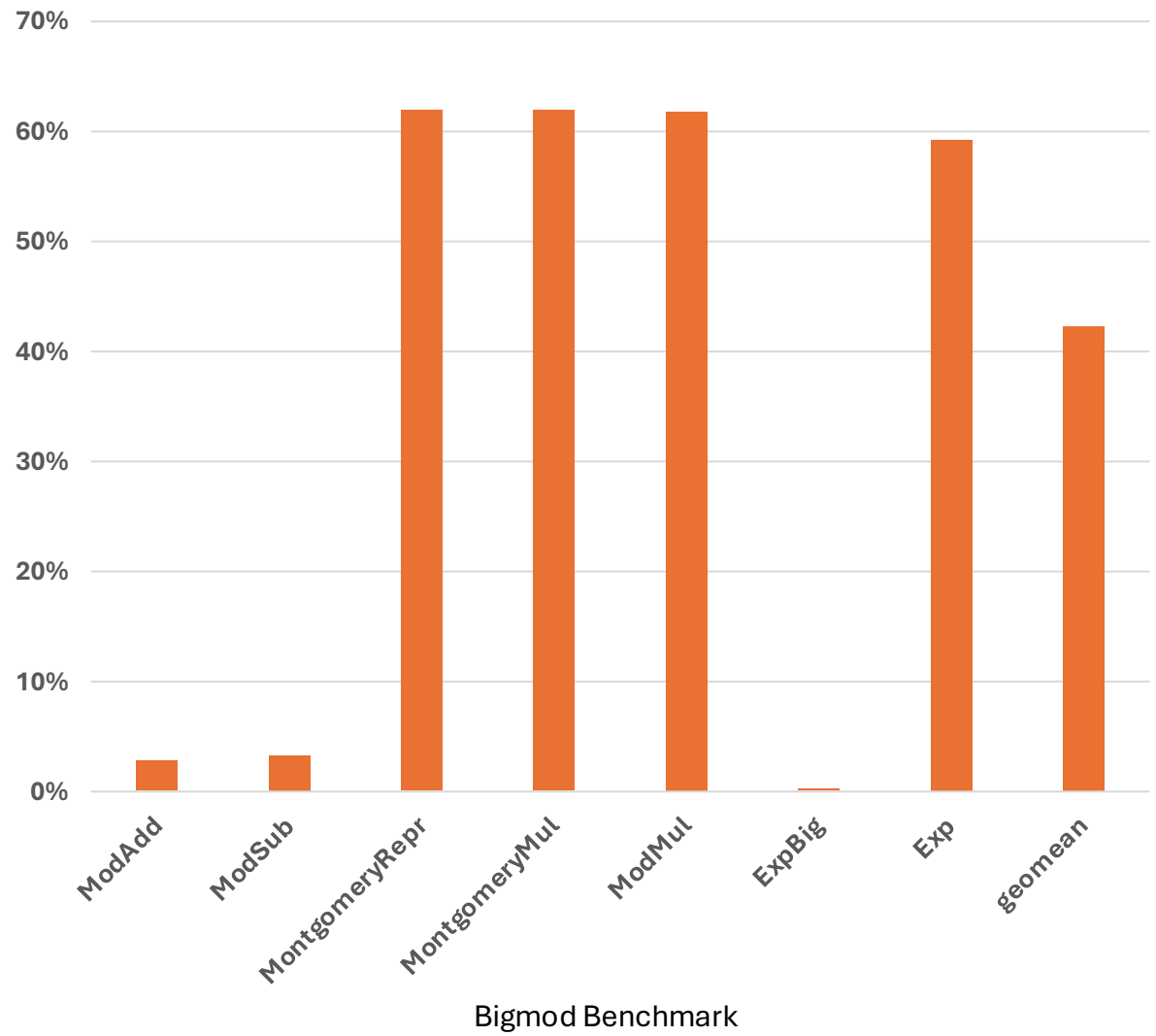


- Memclr and Memmove changes also resulted in significant improvement in ETCD application of OCP:



Crypto Big-Number Arithmetic

- Use cases:
  - RSA Key Generation
    - Montgomery multiplication
    - Generic big-number multiplication
- Issue:
  - Big number multiplication (1KB, 1.5KB, 2KB sizes with a 64-bit word limb) reported bottlenecks on s390x.
- Optimizations:
  - New algorithm with vector instructions for big-num multiplication
- Tools used:
  - Linux Perf
  - GDB & objdump
  - Go Test Suite Micro-benchmarking
- Result: Summary of performance improvement (Go micro-benchmarks)
  - Upto 62% faster in functions using Mul & MontgomeryMul workloads.
  - Upto 43% overall improvement in bigmod workloads.





Performance optimisation in Go compiler & applications
for s390x architecture

Thank You!